Part 1: Maven Web Application with Apache Tomcat

Step 1: Create the Maven Web Application project "MyMavenWebApp01"

Run the following command to generate the project structure using the Maven archetype:

```
mvn archetype:generate -DgroupId=com.example \
    -DartifactId=MyMavenWebApp01 \
    -DarchetypeArtifactId=maven-archetype-webapp \
    -DinteractiveMode=false
```

Step 2: Update pom.xml with the relevant dependencies and plugins

Configure the pom.xml with the Servlet API dependency and the Maven WAR plugin:

```
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>4.0.1</version>
    <scope>provided</scope>
</dependency>

<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-war-plugin</artifactId>
    <version>3.4.0</version>
</plugin>
```

Step 3: Run mvn clean install to compile and generate the WAR file

```
mvn clean install
```

Step 4: Run tree to verify the WAR file was generated

```
tree
```

Step 5: Start the Apache Tomcat Server

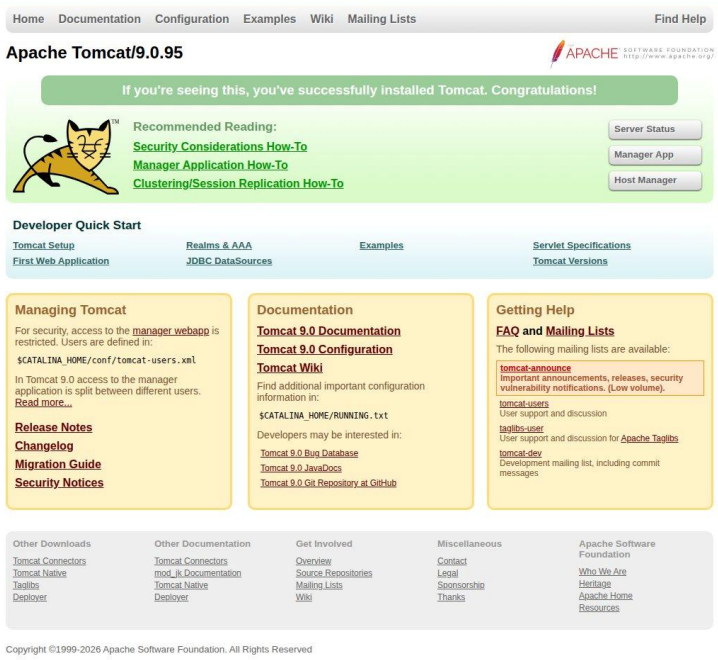
Navigate to the Tomcat binary directory and run the startup script:

```
cd /opt/tomcat/bin
sh startup.sh
```

```
vboxuser@Ubuntu:~/Desktop$ sudo -i
[sudo] password for vboxuser:
Sorry, try again.
[sudo] password for vboxuser:
root@Ubuntu:~#
root@Ubuntu:~# cd /opt
root@Ubuntu:/opt# cd tomcat/bin
root@Ubuntu:/opt/tomcat/bin# sh startup.sh
Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
root@Ubuntu:/opt/tomcat/bin#
```

Step 6: Verify Tomcat is running at localhost:9090

Open a browser and navigate to localhost:9090 to confirm the Apache Tomcat server is running successfully.



Step 7: Update the index.jsp file with your content

Modify the index.jsp file inside src/main/webapp/ with the desired HTML:

```html
<html>
<body>
<h2>Hello World!</h2>
<h2>Someesvaar V</h2>
<h2>1BI23CS210</h2>
</body>
</html>
```

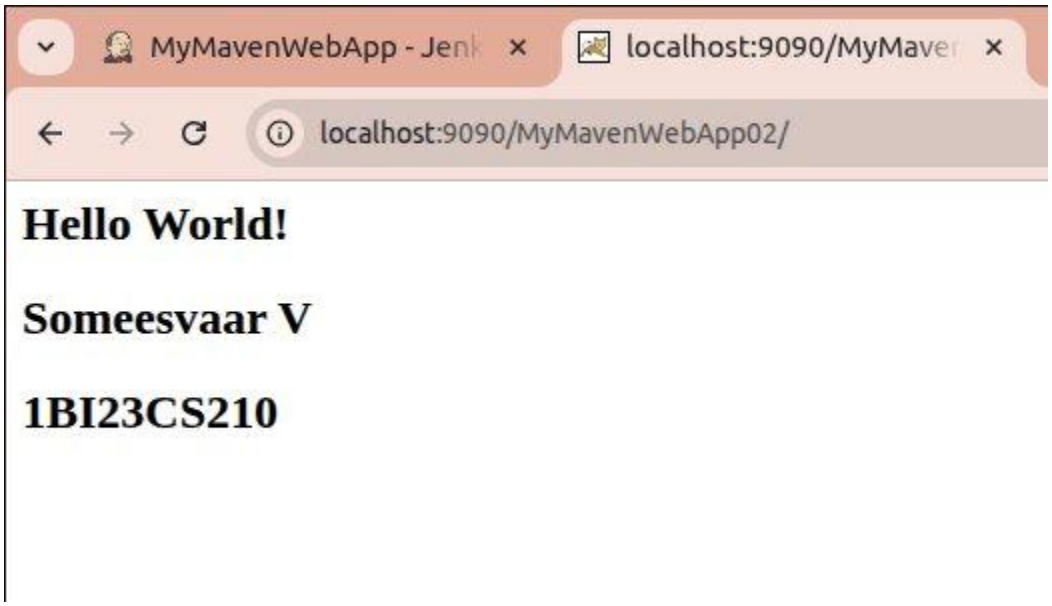
Step 8: Copy the WAR file to the Tomcat webapps directory

Deploy the generated WAR file by copying it to /opt/tomcat/webapps/:

```
cp target/MymavenWebApp01.war /opt/tomcat/webapps/
```

Step 9: Access the deployed web application in the browser

Navigate to localhost:9090/MymavenWebApp01/ in your browser to view the output:



Part 2: Jenkins Pipeline for MyMavenWebApp

Step 1: Open the Jenkins Dashboard and create a New Item

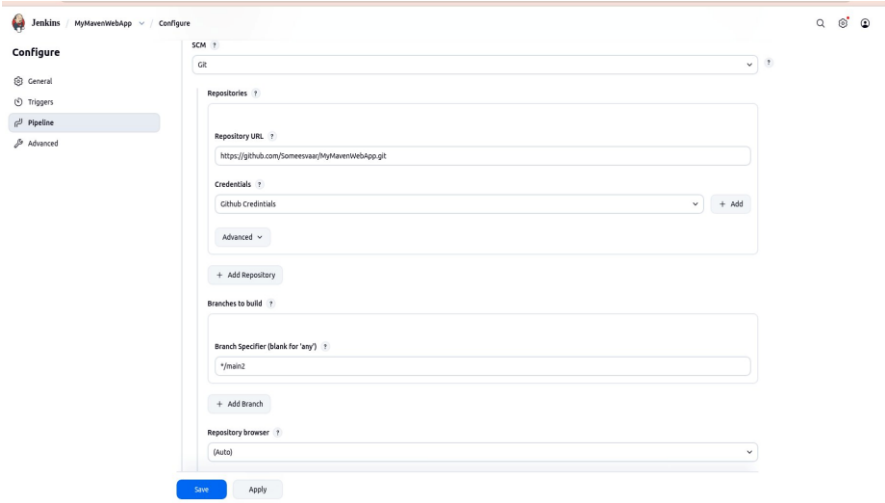
Navigate to the Jenkins dashboard at localhost:8080 and click on "New Item".

Step 2: Name the pipeline "MyMavenWebApp" and select Pipeline type

Enter the project name and select "Pipeline" as the item type, then click OK.

Step 3: Configure the Pipeline — connect to GitHub repository

Set Definition to "Pipeline script from SCM", SCM to "Git", and provide the repository URL with credentials:



Step 4: Create the Jenkinsfile in your project directory

```
cd ~/Devops/MyMavenWebApp
touch Jenkinsfile
```

Step 5: Paste the pipeline script into the Jenkinsfile

The Jenkinsfile defines the full CI/CD pipeline with the following stages:

```
pipeline {
    agent any
    tools {
        maven 'Maven'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main2', url:
'https://github.com/Someesvaar/MyMavenWebApp.git'
            }
```

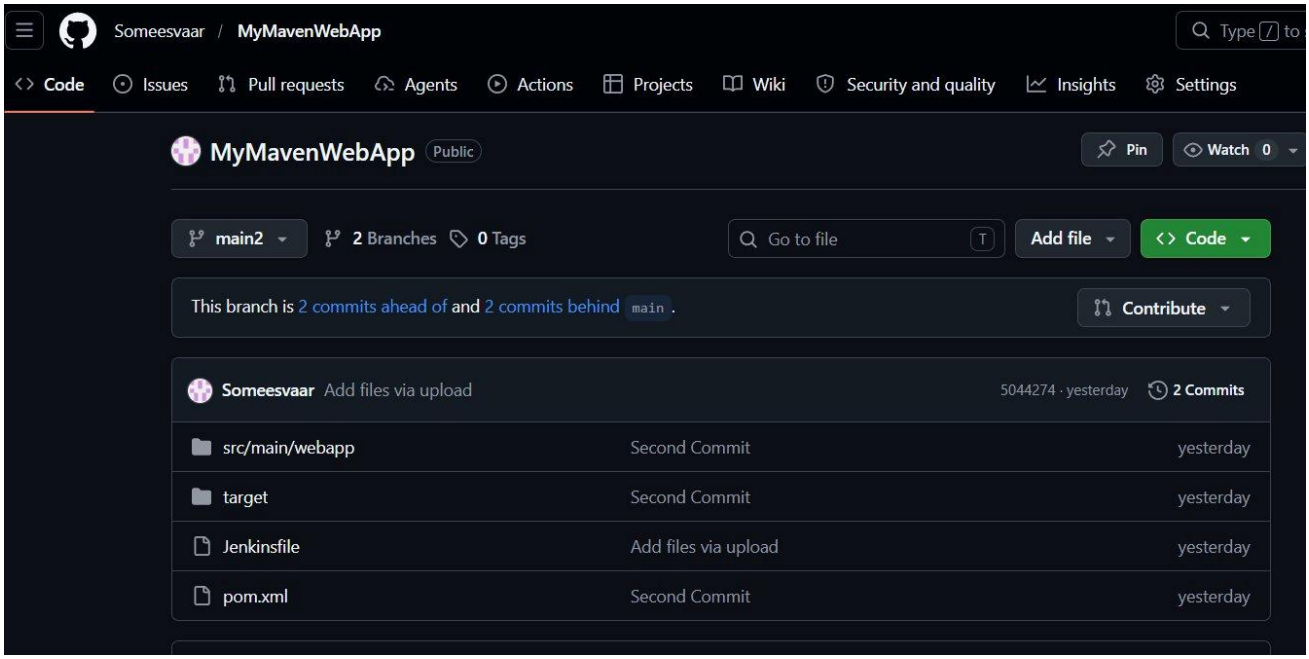
```
        }
        stage('Build') {
            steps { sh 'mvn clean package' }
        }
        stage('Archive') {
            steps {
                archiveArtifacts artifacts: 'target/*.war', fingerprint: true
            }
        }
        stage('Deploy') {
            steps { sh 'mvn clean package' }
        }
    }
    post {
        success { echo 'Build and deployment successful!' }
        failure { echo 'Build failed!' }
    }
}
```

Step 6: Update pom.xml — change Java source/target version to 1.8

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.8.1</version>
    <configuration>
        <source>1.8</source>
        <target>1.8</target>
    </configuration>
</plugin>
```

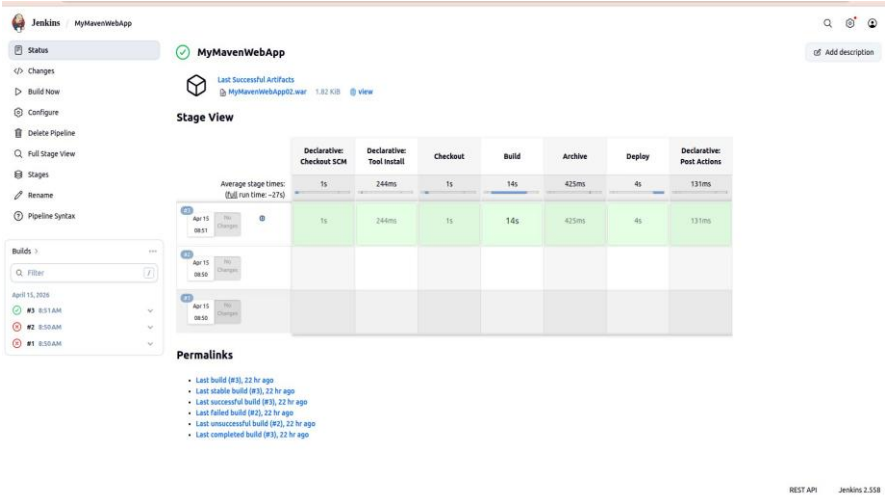
Step 7: Push the Jenkinsfile and updated pom.xml to GitHub

Commit and push the Jenkinsfile to the main2 branch of the GitHub repository:



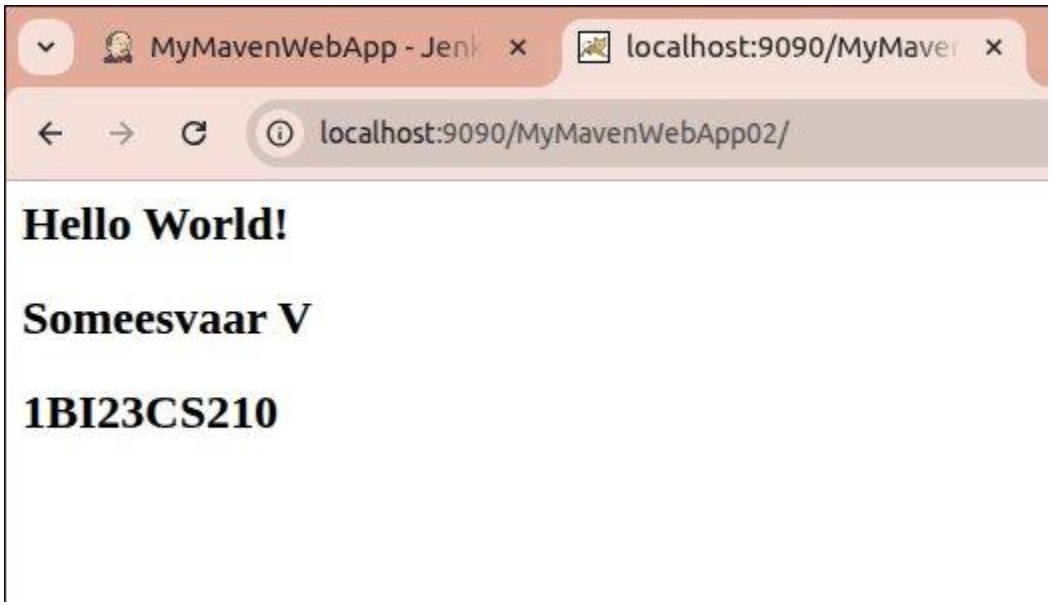
Step 8: Build the pipeline in Jenkins and verify the Stage View

Click "Build Now" in Jenkins and monitor the Stage View for successful completion of all stages:



Step 9: Verify the deployed web application at localhost:9090

After the Jenkins pipeline completes successfully, open the browser and navigate to localhost:9090/MyMavenWebApp02/ to confirm the deployment:



Result

The Maven Web Application was successfully built, packaged as a WAR file, and deployed to the Apache Tomcat server (port 9090) via a Jenkins CI/CD pipeline. The web application is accessible at localhost:9090/MyMavenWebApp02/ and displays the expected output correctly.